Mathématiques et Python

Le langage Python seul ne sait pas faire grand chose dans le domaine mathématique, comme tracer une fonction, calculer des valeurs de fonctions usuelles, réaliser des opérations matricielles,... Cependant, de nombreux modules ont été développés pour pallier ce manque, parmi lesquels in convient de citer :

- scipy
- numpy
- matplotlib

A noter que le module pylab intègre ces trois modules et ipython.

L'objectif de ce document n'est bien entendu pas d'être exhaustif sur ce qu'il est possible de faire avec python et ces modules, mais juste de donner quelques points d'entrée sur ces librairies et de proposer des illustrations par l'exemple de leur utilisation. Dans la mesure du possible, les exemples collent "pas trop loin" du programme officiel de maths MPSI.

1 Ce que l'on peut faire sans les modules...

1.1 Types

Les types de base qui peuvent être utiles dans la suite sont les suivants :

1.1.1 Types numériques

- integer (attention à la division entre entiers !)
- float
- complex : l'imaginaire pur i est noté j en python.

A tout instant, il est possible d'accéder au type d'une variable a en tapant $type(a)$

Toute variable définie avec un type change de type lors d'une nouvelle affectation. On peut aussi changer de type à l'aide des fonctions $int()$, $float()$. L'une des caractéristiques importantes de Python est le typage dynamique. Cependant, si certaines opérations provoquent un changement de type, certaines restent interdites.

1.1.2 Conteneurs

- listes (par exemple $tab = [1, 2, 3, 4, 5]$)
- index (les indices de listes commencent à 0 : par exemple $a[2]$ donne 3)
- slices ($a[1 : 3]$ donne $[2, 3]$)

Le typage dans les listes est faible, on peut combiner différents types numériques (ou non comme des chaînes de caractères, des booléens...)

De nombreuses fonctions sont associées à ces listes (concaténation, recherche de sous-chaînes...).

1.2 Opérateurs élémentaires

Les opérateurs classiques suivantes sont disponibles :

1. +,-,*,/
2. modulo : %
3. exposant : **
4. division entière :// (par exemple 9//2=4)
5. opérateurs de comparaison : ==, !=, <>, <,<=,>,>=
6. opérateurs d'affectation : =,+=,-=,*=,/=,%=,**=,//=
7. les opérateurs bit à bit : & (et),|| (ou), ˆ (XOR),˜ (complément à 1), <<(décalage à gauche), >> (décalage à droite)
8. opérateurs logiques : and, or, not
9. opérateurs d'appartenance (sur des types comme des chaînes) : in, not in
10. opérateurs d'identité : is, is not

1.3 La librairie standard math

Pour disposer des fonctions mathématiques usuelles, la librairie d'origine du python se nomme math. On peut alors d'importer juste les fonctions nécessaires par from math import cos, log ou toutes les fonctions mathématiques par from math import *. Dans le premier cas l'inconvénient est qu'il faut savoir à l'avance les fonctions utilisées par la suite, dans le deuxième cas on risque de surcharger inutilement la mémoire.

A noter que pour manipuler des complexes, il faut importer le module cmath en plus du module math (par exemple pour réaliser des produits de complexes).

1.4 Un exemple : calcul d'intégrales

Pour illustrer les capacités de base de Python, nous proposons de calculer de manière numérique la valeur de $I = \int_a^b f(x)dx$, en utilisant trois méthodes classiques :

- la méthode des rectangles :$I \approx \sum_{i=0}^{n-1}(x_{i+1} - x_i) f\left(\frac{x_i + x_{i+1}}{2}\right)$
- la méthode des trapèzes : $I \approx h\left[\frac{f(a)+f(b)}{2} + \sum_{i=1}^{n-1} f(x_i)\right]$
- la méthode de Simpson : $I \approx \frac{h}{6}\left[f(a) + f(b) + 4\sum_{i=0}^{n-1} f(x_{2i+1}) + 2\sum_{i=1}^{n-1} f(x_{2i})\right]$ avec $h = \frac{b-a}{n}$ et $x_k = a + k\frac{h}{2}$

et où $(x_0 \cdots x_n)$ est une subdivision régulière de l'intervalle $[a, b]$ de pas h

Le code 1 donne le source python permettant de réaliser ces trois approximations.

```
# -*- coding: utf-8 -*-
def rectangles(f,a,b,n):
#Methode des rectangles
    S=0
    for i in xrange(0,n):
        xi=a+(b-a)*i/float(n)
        xj=a+(b-a)*(i+1)/float(n)
        S+= f((xi+xj)/2.0)*(xj-xi)
    return S

def trapezes(f,a,b,n):
#Methode des trapezes
    S=0
    for i in xrange(0,n):
        xi=a+(b-a)*i/float(n)
        xj=a+(b-a)*(i+1)/float(n)
        S+= (f(xi)+f(xj))/2.0*(xj-xi)
    return S

def simpson(f,a,b,n):
#Methode de Simpson
    S=0
    for i in xrange(0,n):
        xi=a+(b-a)*i/float(n)
        xj=a+(b-a)*(i+1)/float(n)
        S+= (xj-xi)*(f(xi)+4*f((xi+xj)/2.0)+f(xj))/6.0
    return S

def fn(x):
#fonction a integrer
    return 4.0/(1+(x-3)*(x-3))

def main ():
        print "par rectangles : ", rectangles(fn,0.,5.,100);
        print "par trapèzes : ",trapezes(fn,0.,5.,100);
        print "par Simpson : ",simpson(fn,0.,5.,100);


main()
```

Listing 1 – Approximation numérique d'une intégrale par trois méthodes classiques

1.5 Un autre exemple autour des suites

Prenons un exemple classique, celui du calcul d'une estimation du nombre d'or à l'aide de la suite de Fibonacci. Le code 2 présente le calcul des n premiers termes de la suite de Fibonacci $u_0 = 1, u_1 = 1$ et $u_n = u_{n-1} + u_{n-2}, n \geq 2$ ainsi que la valeur absolue de la différence avec le nombre d'or $\frac{1+\sqrt{5}}{2}$.

```
# -*- coding: utf-8 -*-
def fibonacci(n):
        a = b = 1.
        for i in range(n):
                a, b = a + b, a
                print abs((a/b)-(1+5**0.5)/2)
        return b



def main():
        fibonacci(30)


main()
```

Listing 2 – Calcul approché du nombre d'or

Exercice 1 *Proposer un code permettant de calculer ces mêmes quantités de manière récursive.*

Exercice 2 *Proposer un code permettant de calculer la somme des éléments d'une suite quelconque indicés par un ensemble d'entiers J*

1.6 Un dernier exemple : zéro d'une fonction

Le code 3 présente un calcul simple d'un zéro d'une fonction dans un intervalle donné, en utilisant une approche dichotomique.

```
# -*- coding: utf-8 -*-

def f(x):
    return x**2  +20*x -12

def zero(f,a,b):
    if f(a)*f(b)>0:
        print('La fonction ne s''annule pas dans l''intervalle ['+str(a)+','+str(b)
            +']')
        return 0
    while(abs(a-b)>1e-3):

        m=(a+b)/2.
        print m
        if f(m)*f(a)>0:
            a=m
        else:
            b=m
    print('la solution de f(x)=0 est '+str(m))
    return m



print(zero(f,-10.,10.))
```

Listing 3 – Zéro d'une fonction sur un intervalle par dichotomie.

Exercice 3 *Produire un code qui calcule le zéro d'une fonction en utilisant la méthode de Newton (algorithme 1) :*

Algorithm 1: Méthode de Newton

Entrées: N, ϵ, f, f_p, x_0

n$\leftarrow 0$

$x_n \leftarrow x_0$

répéter

  $x_n \leftarrow x_n - \frac{f(x_n)}{f_p(x_n)}$

  $n \leftarrow n + 1$

  si $f_p(x_n) < \epsilon$ **alors**

    Division par zero

  fin

jusqu'à $\left\| \frac{f(x_n)}{f_p(x_n)} \right\| < \epsilon \ OU \ n > N$;

Exercice 4 *Calculer une approximation de π en utilisant par exemple les deux résultats classiques :*

$$\frac{\pi^2}{6} = \sum_{n=1}^{\infty} \frac{1}{n^2} \quad et \quad \frac{\pi}{2} = \sum_{n=1}^{\infty} \frac{4n^2}{4n^2 - 1}$$

2 ... Et là où ça va mieux : utilisation des librairies

Python présente l'avantage de recourir aux modules pour le développement de fonctions ou d'ensembles de fonctionnalités spécifiques. Cela permet une grande flexibilité et une dynamique de développement importante. Parmi ces modules nous nous intéressons particulièrement dans la suite à Numpy, Scipy et Matplotlib. Suivant la distribution de Python choisie, l'ensemble de ces modules, avec d'autres, est automatiquement installé lors de l'installation de Python. Si ce n'est pas le cas il y a toujours la possibilité de les installer a posteriori.

2.1 Présentation rapide des modules

Ces modules fournissent un ensemble d'objets ainsi qu'un groupes de fonctions permettant de manipuler nombre d'objets de façon simple et très performantes dans le cadre du calcul scientifique. Voici la description donnée par le site officiel de Numpy (http ://www.scipy.org, numpy.scipy.org)

SciPy is a collection of mathematical algorithms and convenience functions built on the Numpy extension for Python. It adds significant power to the interactive Python session by exposing the user to high-level commands and classes for the manipulation and visualization of data. With SciPy, an interactive Python session becomes a data-processing and system-prototyping environment rivaling sytems such as MATLAB, IDL, Octave, R-Lab, and SciLab. NumPy is the fundamental package needed for scientific computing with Python. It contains among other things :

- a powerful N-dimensional array object - sophisticated (broadcasting) functions tools for integrating C/C++ and Fortran code
- useful linear algebra, Fourier transform, and random number capabilities.

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data types can be defined. This allows NumPy to seamlessly and speedily integrate with a wide variety of databases.

Scipy est un ensemble qui comprend de nombreux modules utiles pour des scientifiques :

- cluster : information theory functions (currently, vq and kmeans)
- weave : compilation of numeric expressions to C++ for fast execution
- fftpack : fast Fourier transform module based on fftpack and fftw when available
- ga : genetic algorithms
- io : reading and writing numeric arrays, MATLAB .mat, and Matrix Market .mtx files
- integrate : numeric integration for bounded and unbounded ranges. ODE solvers.
- interpolate : interpolation of values from a sample data set.
- optimize : constrained and unconstrained optimization methods and root-finding algorithms
- signal : signal processing (1-D and 2-D filtering, filter design, LTI systems, etc.)
- special : special function types (bessel, gamma, airy, etc.)
- stats : statistical functions (stdev, var, mean, etc.)
- linalg : linear algebra and BLAS routines based on the ATLAS implementation of LAPACK
- sparse : Some sparse matrix support. LU factorization and solving Sparse linear systems.

Enfin Matplotlib permet de visualiser en 2D des données.

2.2 Quelques exemples de Numpy

Numpy ajoute le type array qui est similaire à une liste (list) avec la condition supplémentaire que tous les éléments sont du même type.

Le code 4 présente quelques exemples d'instantiation de matrices simples.

```
# -*- coding: utf-8 -*-
import numpy as np

# tableau 1D
a1 = np.array([1, 2, 3, 4], float)
print a1
#tableau 2D
a2=np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
print a2

#matrices de 1
un=np.ones(5)
print un

#matrice diagonale
d = np.diag(a1)
print d

#matrice bande
d1 = np.diag(a1,-1)
print d1

#matrice à coefficients aléatoires dans [0,1]
r = np.random.rand(3,3)
print r

# Identité
i = np.eye(5)
print i

# Matrice nulle
z = np.zeros(5)
print z
```

Listing 4 – Définitions de matrices

Les opérations classiques sur la matrices sont disponibles à l'aide de numpy : addition, multiplication par un scalaire, produit matriciel...Le code 5 présente quelques exemples de ces opérations.

```
# -*- coding: utf-8 -*-
import numpy as np

A = np.random.rand(3,3)
B=np.diag([1.,2.,3.])
v = np.array([3.,4.,5.], float)

# addition
C1 = A+B
C2 = 2.+A

#multiplication
D1 = 2*A          #coefficients de A multipliés par 2
D2 = B**3         #coefficients de B à la puissance 3
D3 = A*B          # multiplication terme à terme
D4 = np.dot(A,B)# multiplication matricielle
D5 = np.dot(A,v)#produit matrice/vecteur
D6 = np.kron(A,B)#produit de Kronecker

#test
E1 = A<B#renvoie une matrice de booléens effectuant le test
bo = np.array([1,0.,0], bool)
E2=B[bo]#extrait les éléments de B qui correspondent à la valeur vraie de bo
E3=A[A>0.5]
```

Listing 5 – Opérations sur les matrices

Bien entendu, numpy permet facilement de faire du calcul numérique matriciel : calcul du rang d'une matrice, inversion d'une matrice, résolution de systèmes linéaires. A titre d'exemple, le code 6 présente quelques possibilités offertes par le module.

```python
# -*- coding: utf-8 -*-
import numpy as np
import numpy.linalg as nl

A = np.random.rand(3,3)
b = np.array([3.,4.,5.], float)

#Transposition d'une matrice
Aprime=A.transpose()

#Rang d'une matrice
r = np.rank(A)

#Inverse d'une matrice
Ainv = nl.inv(A) #attention tester si A est inversible...

#Résolution de systèmes linéaires
x = nl.solve(A,b)

#calcul des éléments propres
nl.eig(A) #valeurs propres, matrice de passage

#Calcul de normes
n1 = nl.norm(A,np.inf);
n2 = nl.norm(A,-np.inf);
n3 = n1 = nl.norm(A,2);
n4 = n1 = nl.norm(A,'fro');
```

Listing 6 – Un peu d'algèbre linéaire avec numpy

Exercice 5 *Proposer un code qui code la décomposition de Cholesky d'une matrice A. Comparer avec l'appel à numpy.linalg.cholesky. Pour rappel, l'algorithme de Cholesky est le suivant :*

Algorithm 2: Méthode de Cholesky

pour $k \in \{1 \cdots n\}$ **faire**

$$a_{kk} \leftarrow \left(a_{kk} - \sum_{p=1}^{k-1} a_{kp}^2\right)^2$$

pour $i \in \{k+1 \cdots n\}$ **faire**

$$a_{ik} \leftarrow \frac{1}{a_{kk}}\left(a_{ik} - \sum_{p=1}^{k-1} a_{ip}a_{kp}\right)$$

fin

fin

Notons que numPy propose de nombreux autres atouts, que nous vous conseillons de découvrir dans la documentation de ce module. A titre d'exemple, citons la classe poly1d qui gère les polynômes à une variable, documentée comme suit :

```
class numpy.poly1d(c_or_r, r=0, variable=None)[source]
A one-dimensional polynomial class.
A convenience class, used to encapsulate natural operations on polynomials so that
    said operations may take on their customary form in code (see Examples).

Parameters :
c_or_r : array_like
The polynomials coefficients, in decreasing powers, or if the value of the second
    parameter is True, the polynomials roots (values where the polynomial evaluates
     to 0). For example, poly1d([1, 2, 3]) returns an object that represents ,
    whereas poly1d([1, 2, 3], True) returns one that represents .
r : bool, optional
If True, c_or_r specifies the polynomials roots; the default is False.
variable : str, optional
Changes the variable used when printing p from x to variable (see Examples).
Examples

Construct the polynomial :
>>> p = np.poly1d([1, 2, 3])
>>> print np.poly1d(p)
   2
1 x + 2 x + 3
Evaluate the polynomial at :
>>> p(0.5)
4.25
Find the roots:
>>> p.r
array([-1.+1.41421356j, -1.-1.41421356j])
>>> p(p.r)
array([ -4.44089210e-16+0.j,  -4.44089210e-16+0.j])
These numbers in the previous line represent (0, 0) to machine precision

Show the coefficients:
>>> p.c
array([1, 2, 3])
Display the order (the leading zero-coefficients are removed):
>>> p.order
2
Show the coefficient of the k-th power in the polynomial (which is equivalent to p.
    c[-(i+1)]):
>>> p[1]
2
Polynomials can be added, subtracted, multiplied, and divided (returns quotient and
     remainder):
>>> p * p
poly1d([ 1,  4, 10, 12,  9])
>>> (p**3 + 4) / p
(poly1d([  1.,   4.,  10.,  12.,   9.]), poly1d([ 4.]))
asarray(p) gives the coefficient array, so polynomials can be used in all functions
     that accept arrays:
```

Listing 7 – Documentation de la classe poly1d

```
>>> p**2 # square of polynomial
poly1d([ 1,  4, 10, 12,  9])
>>> np.square(p) # square of individual coefficients
array([1, 4, 9])
The variable used in the string representation of p can be modified, using the
    variable parameter:

>>> p = np.poly1d([1,2,3], variable='z')
>>> print p
   2
1 z + 2 z + 3
Construct a polynomial from its roots:

>>> np.poly1d([1, 2], True)
poly1d([ 1, -3,  2])
This is the same polynomial as obtained by:

>>> np.poly1d([1, -1]) * np.poly1d([1, -2])
poly1d([ 1, -3,  2])
Attributes

coeffs
order
variable
Methods

__call__(val)
deriv([m])       Return a derivative of this polynomial.
integ([m, k])    Return an antiderivative (indefinite integral) of this polynomial.
```

Listing 8 – Documentation de la classe poly1d : suite

Exercice 6 *Proposer un code, utilisant la classe poly1d, et codant les polynômes de Legendre :*

$$P_0(x) = 1, P_1(x) = x,$$

et pour tout entier $n > 0$

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x).$$

2.3 Quelques exemples de Scipy

Scipy est construit à partir de Numpy, ce qui signifie qu'il faut avoir le module Numpy pour faire fonctionner le module Scipy. En effet nombre de fonctions ainsi que le type 'ndarray' de Scipy sont en fait ceux définis dans Numpy.

2.3.1 Intégration numérique

Scipy propose une série de classes pour l'intégration. Cet ensemble se trouve regroupé dans le sous-module *scipy.integrate*. L'intégration peut se faire sur un intervalle, à partir d'un échantillon de points ou encore servir à résoudre des équations différentielles (cf. paragraphe 2.3.2)

Le code 9 reprend le calcul de l'intégrale décrit dans le paragraphe 1.4, mais en utilisant trois fonctions fournies par la librairie

```
# -*- coding: utf-8 -*-
from numpy import *
from scipy import integrate


def fn(x):
#fonction a integrer
    return 4.0/(1+(x-3)*(x-3))

def main ():
        print "par Scipy : ", integrate.quad(fn,0,5)
        print "Romberg par Scipy : ", integrate.romberg(fn,0,5)
        #Subdivision de l'intervalle par pas régulier
        x = linspace(0,5,1000)
        y=fn(x)
        print "trapezes par Spicy" , integrate.trapz(y, x, dx = 0.1)

main()
```

Listing 9 – Approximation numérique d'une intégrale en utilisant Spicy

2.3.2 Résolution d'une équation différentielle ordinaire

On souhaite par exemple résoudre l'équation différentielle $\frac{d^2y}{dt^2} = ay + b\frac{dy}{dt}$ pour $t \in [0, 10]$ et une condition initiale sur y et sa dérivée. Les modules d'intégration de Scipy (et plus précisément *odeint*) permettent de trouver y et, en prenant un peu d'avance sur l'affichage (cf. section 2.4), on peut tracer la fonction résultat. Le code 10 propose une solution à ce problème.

```
# -*- coding: utf-8 -*-
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt

#dérivée de y (en tant que tableau : y[0] est la fonction, y[1] la dérivée)
def deriv(y,t):
        a = -2.0
        b = -0.1
        return np.array([ y[1], a*y[0]+b*y[1] ])

tps = np.linspace(0.0,10.0,1000)

#valeurs initiales de y et de sa dérivée
yinit = np.array([0.0005,0.2])
y = odeint(deriv,yinit,tps)
plt.figure()
plt.plot(tps,y[:,0])
plt.xlabel('t')
plt.ylabel('y')
plt.show()
```

Listing 10 – Résolution par intégration d'une équation différentielle ordinaire en utilisant Spicy

2.3.3 Interpolation

Scipy possède un module d'interpolation assez complet, qui comprend plusieurs méthodes d'interpolation définies sous formes de classes. Il est possible d'utiliser des interpolations linéaire ou cubique par exemple. Le code 11 montre quelques appels de ces méthodes. Notons qu'il est nécessaire d'instancier la classe pour l'utiliser. La figure 1 présente le résultat graphique obtenu.

```
# -*- coding: utf-8 -*-
import scipy as sp
import numpy as np
from scipy.interpolate import interp1d
from matplotlib.pyplot import *
x_measure = np.linspace(0.,1,10)
bruit = np.random.uniform(-0.1,0.1,10)
y_measure = np.sin(2 * np.pi * x_measure) +np.tan(2 * np.pi * x_measure) + bruit
# instanciations de la classe interpolation
interp_lin = interp1d(x_measure,y_measure)
interp_cubic = interp1d(x_measure,y_measure,kind='cubic')
interp_quad = interp1d(x_measure,y_measure,kind='quadratic') #
x_computed = np.linspace(0,1.,100)
y_int_lin = interp_lin(x_computed)
y_int_cub = interp_cubic(x_computed)
y_int_quad = interp_quad(x_computed)

import matplotlib.pyplot as plt
plt.plot(x_measure,y_measure,'o',x_computed,y_int_lin,'-', x_computed, y_int_cub,'
    --', x_computed, y_int_quad,'*')
plt.legend(['data', 'linear', 'cubic', 'quad'], loc='best')
plt.show()
```

Listing 11 – interpolation par plusieurs méthodes disponibles dans Spicy

2.4 Quelques exemples de Matplotlib

Le module Matplotlib, comme son nom l'indique, s'occupe du tracé graphique. Nous avons déjà vu un exemple d'utilisation de ce module dans la partie interpolation. Le code 12 et la figure 2 donnent de nouveaux exemples, en illustrant certaines possibilités (titres, labels, types de courbes, couleurs...), tandis que le code 13 et la figure 3 démontrent qu'il est possible d'afficher simultanément plusieurs graphes sur une même figure.

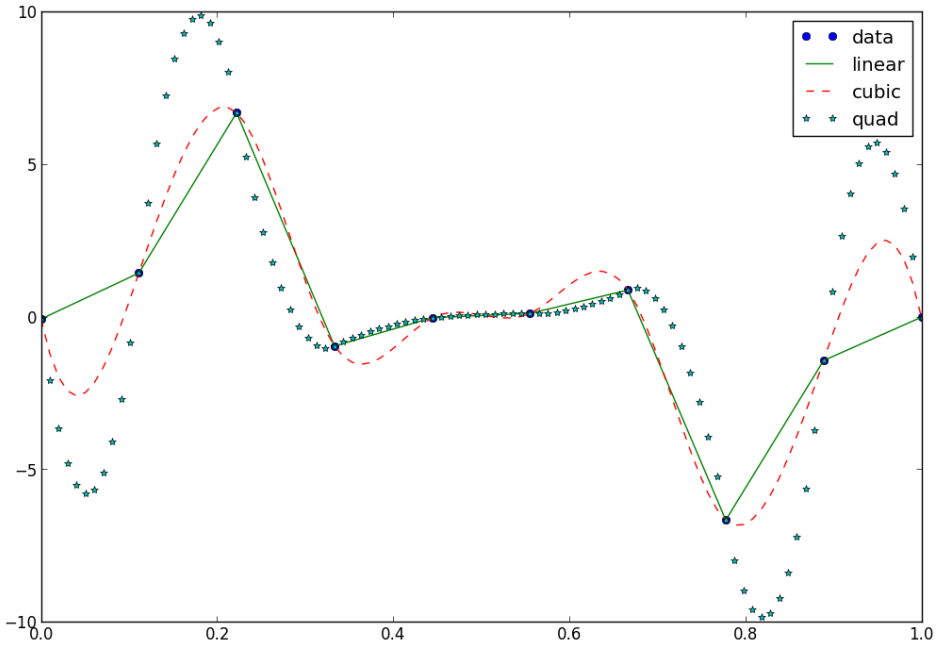

FIGURE 1 – Tracé des interpolants

```
# -*- coding: utf-8 -*-

import matplotlib.pyplot as plt
import numpy as np


t1=np.linspace(1,5,10)
t2=np.linspace(1,5,20)
plt.plot(t1, t1, 'r--', t1, t1**2, 'bs', t2, np.log(t2)**3, 'g^-')
plt.xlabel("Abcisses")
plt.ylabel('fonctions')
plt.legend(['courbe 1','courbe 2','courbe 3'], loc='best')
plt.title("Zoulies courbes")
plt.show()
```

Listing 12 – Quelques possibilités de base de matplotlib

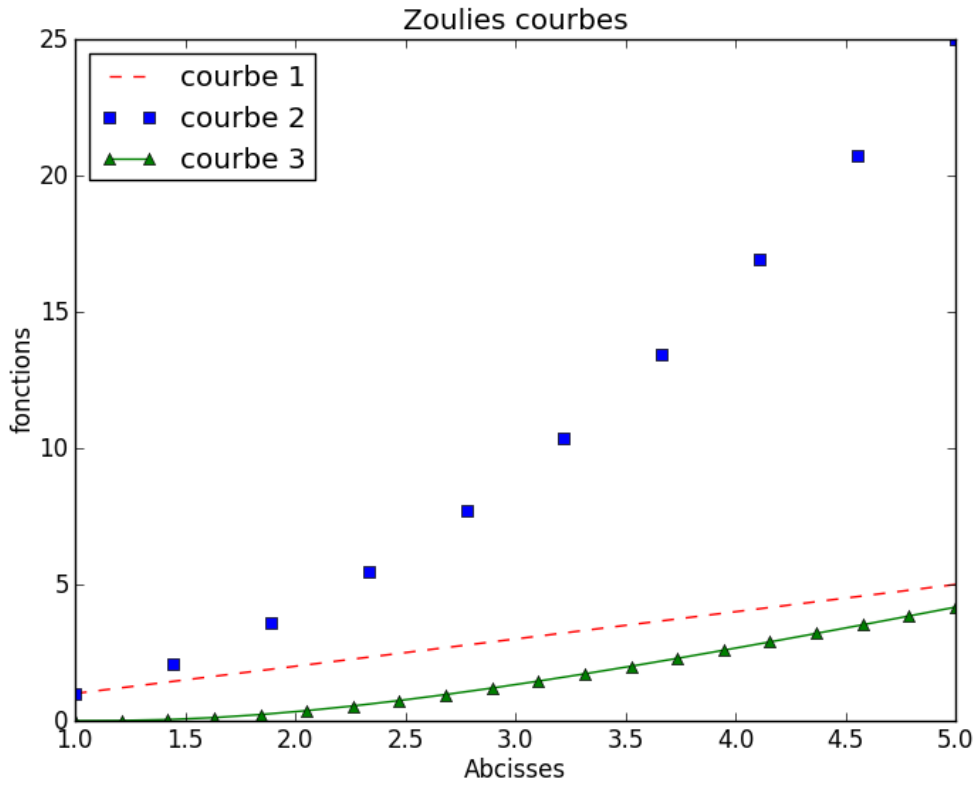

FIGURE 2 – résultat du code 12

```
import numpy as np
import matplotlib.pyplot as plt
def f(t):
        return np.exp(-t) * np.cos(2*np.pi*t)
def g(t):
        return np.exp(t) * np.sin(2*np.pi*t)
def h(t):
        return np.cos(2*np.pi*t)**3

t1 = np.arange(0.0, 5.0, 0.1)
t2 = np.arange(0.0, 5.0, 0.02)
plt.figure(1)

plt.subplot(221)
plt.xlabel("Abcisses")
plt.plot(t1, f(t1), 'bo', t2, f(t2), 'k')
plt.xlabel("Abcisses")
plt.ylabel("f")

plt.subplot(222)
plt.plot(t2, g(t2), 'r--')
plt.xlabel("Abcisses")
plt.ylabel("g")

plt.subplot(223)
plt.plot(t1, h(t1), 'b-')
plt.xlabel("Abcisses")
plt.ylabel("h")

plt.show()
```


Listing 13 – Affichage de plusieurs figures

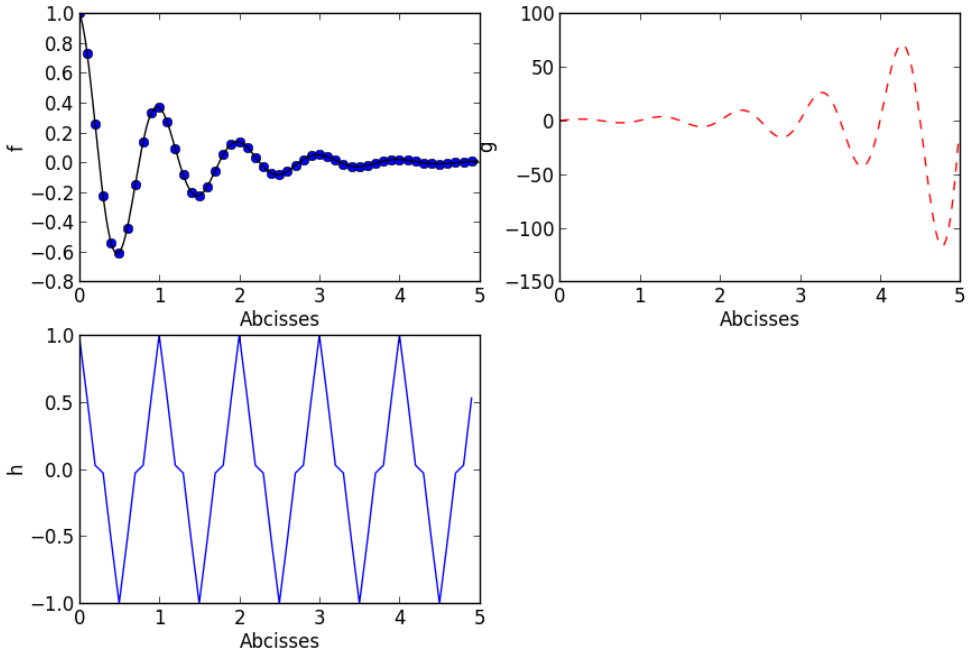

FIGURE 3 – résultat du code 13

Toutes les courbes peuvent bien sur être tracées avec le module matplotlib. Le code 14 et la figure 4 donnent quelques exemples de courbes paramétrées classiques.

```
from math import *
import numpy as np
import matplotlib.pyplot as plt

# Valeurs du paramètres pour les points tracés le long de la courbe
lt = np.linspace(0,2*pi,100)


plt.figure(1)

plt.subplot(221)
plt.plot([3+1.5*cos(t)*(1+cos(t)) for t in lt], [sin(t)*(1+cos(t)) for t in lt],'g^
    ')
plt.title("Cardioide")

plt.subplot(222)
plt.plot([cos(t)**3 for t in lt], [sin(t)**3 for t in lt],'k')
plt.title("Astroide")

plt.subplot(223)
lt = np.linspace(0,10*pi,100)
plt.plot([3*(t-sin(t)) for t in lt], [3*(1-cos(t)) for t in lt],'r')
plt.title("cycloide")

plt.subplot(224)
lt = np.linspace(0,2*pi,100)
plt.plot([2*sin(t)**2*cos(t) for t in lt], [2*sin(t)*cos(t)**2 for t in lt],'g-')
plt.title("quadrifolium")

plt.show()
```

Listing 14 – Quelques courbes paramétrées

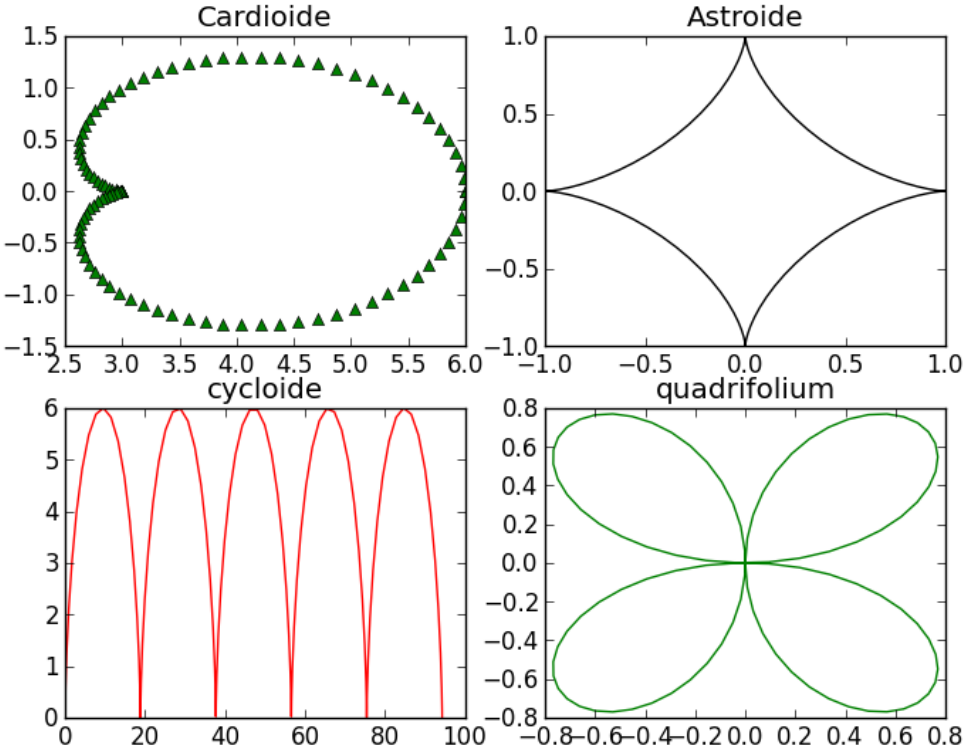

FIGURE 4 – résultat du code 14